

Technical Module Overview — pqc_crypto_module v0.1.0

Disclaimer: Prepared for FIPS 140-3 evaluation, not currently validated.

1. Architecture

pqc_crypto_module is a standalone Rust crate (`crates/pqc_crypto_module/`) that encapsulates all cryptographic operations for the Cerulean Ledger DLT platform. It is compiled as a library (`rlib`) and linked into the main application binary. The module has no runtime dependencies outside of the OS-provided entropy source.

Language: Rust (Edition 2021) **Type:** Software cryptographic module **Target security level:** FIPS 140-3 Level 1

2. Cryptographic Boundary

The boundary consists of **11 source files** in `crates/pqc_crypto_module/src/`:

File	Responsibility
<code>lib.rs</code>	Crate root; module re-exports
<code>api.rs</code>	Single public entry point for all approved operations
<code>mldsa.rs</code>	ML-DSA-65 key generation, signing, verification
<code>mlkem.rs</code>	ML-KEM-768 key encapsulation and decapsulation
<code>hashing.rs</code>	SHA3-256 hashing
<code>rng.rs</code>	CSPRNG wrapper with continuous health test
<code>self_tests.rs</code>	Known Answer Tests (KATs) for all approved algorithms
<code>approved_mode.rs</code>	State machine and approved-mode guards
<code>types.rs</code>	Typed cryptographic wrappers with <code>ZeroizeOnDrop</code>
<code>errors.rs</code>	<code>CryptoError</code> enum
<code>legacy.rs</code>	Non-approved algorithms (runtime-gated, compile-time excludable)

Boundary enforcement is provided by the Rust crate module system. External code can only access items explicitly marked `pub` in the crate's API. An integration test (`tests/api_boundary.rs`) verifies that no cryptographic operation succeeds without passing through the approved API.

3. Public API

The module exposes **8 public functions** through `pqc_crypto_module::api`:

Function	Algorithm	Operation
<code>initialize_approved_mode()</code>	All	Run self-tests; transition to Approved state
<code>generate_mldsa_keypair()</code>	ML-DSA-65	Generate signing keypair
<code>sign_message(sk, msg)</code>	ML-DSA-65	Produce digital signature
<code>verify_signature(pk, msg, sig)</code>	ML-DSA-65	Verify digital signature
<code>sha3_256(data)</code>	SHA3-256	Compute 256-bit hash
<code>generate_mlkem_keypair()</code>	ML-KEM-768	Generate encapsulation keypair
<code>mlkem_encapsulate(pk)</code>	ML-KEM-768	Encapsulate shared secret
<code>mlkem_decapsulate(sk, ct)</code>	ML-KEM-768	Decapsulate shared secret

An additional utility function `random_bytes(n)` provides approved-mode-guarded CSPRNG output.

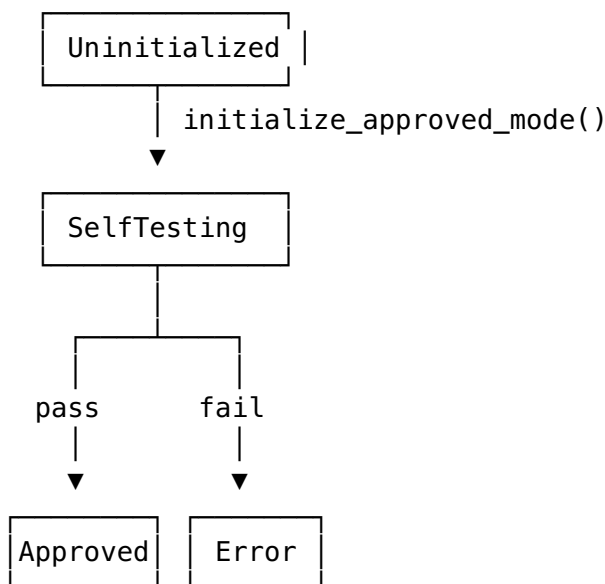
All functions except `initialize_approved_mode()` require the module to be in Approved state and return `CryptoError::NotInitialized` otherwise.

4. Approved Algorithms

Algorithm	Standard	Key Sizes	Output Sizes
ML-DSA-65	FIPS 204	PK: 1952 B, SK: 4032 B	Signature: 3309 B
ML-KEM-768	FIPS 203	Per spec (placeholder impl.)	Shared secret: 32 B
SHA3-256	FIPS 202	N/A	Digest: 32 B

Implementation note: ML-KEM-768 is currently a structural placeholder using SHA3-based key derivation. It will be replaced with a FIPS 203-compliant implementation when a suitable Rust library is available. The public API will not change.

5. State Machine



- **Uninitialized:** No cryptographic operations available. Only `initialize_approved_mode()` may be called.
- **SelfTesting:** KATs executing. No external operations permitted.
- **Approved:** All approved-mode API functions available. Non-approved algorithms blocked.
- **Error:** All operations blocked. Module must be restarted (process restart).

6. Self-Tests

Power-on self-tests executed during `initialize_approved_mode()`:

Test	Algorithm	Type
ML-DSA-65 KAT	ML-DSA-65	Sign known message, verify signature
ML-KEM-768 KAT	ML-KEM-768	Encapsulate/decapsulate round-trip
SHA3-256 KAT	SHA3-256	Hash known input, compare to expected output
RNG health test	CSPRNG	

Test	Algorithm	Type
		Verify non-repeating output (continuous test)

Failure of any self-test transitions the module to Error state.

7. Key Management

- All key types (`MldsaPrivateKey`, `MldsaPublicKey`, `MlKemPrivateKey`, etc.) are newtype wrappers.
- Private key types derive `ZeroizeOnDrop` from the `zeroize` crate, ensuring memory is zeroed when the key goes out of scope.
- Keys are generated within the boundary and returned to the caller as opaque typed values.
- No key serialization to persistent storage is performed by the module; key persistence is the caller's responsibility.
- Key lifecycle: Generate -> Use -> Drop (automatic zeroization).

8. Zeroization

Type	Zeroization Method
<code>MldsaPrivateKey</code>	<code>ZeroizeOnDrop</code> (automatic on drop)
<code>MlKemPrivateKey</code>	<code>ZeroizeOnDrop</code> (automatic on drop)
<code>MlKemSharedSecret</code>	<code>ZeroizeOnDrop</code> (automatic on drop)
Intermediate buffers	Scoped within function; zeroed by Rust ownership semantics

Integration test `tests/key_zeroization.rs` verifies zeroization behavior.

9. Non-Approved Legacy Handling

Algorithm	Purpose	Gating
Ed25519	Legacy signature verification	<code>ensure_not_approved()</code> — blocked in Approved state
SHA-256	Legacy block hashing	<code>ensure_not_approved()</code> — blocked in Approved state
HMAC-SHA256	Legacy MAC operations	<code>ensure_not_approved()</code> — blocked in Approved state

The approved-only Cargo feature flag excludes the entire legacy module at compile time via `compile_error!`, producing a binary with zero non-approved algorithm code.

10. Test Coverage

- **1500+ tests** across 12 test suites.
- **Test categories:** API boundary, approved vs. legacy separation, no-fallback enforcement, self-test execution, key zeroization, FIPS readiness.
- **100% boundary compliance:** verified by integration tests.

11. Dependencies

Crate	Version	Role
pqcrypto-mldsa	0.1.2	ML-DSA-65 implementation
sha3	0.10	SHA3-256
rand / rand_core	0.8 / 0.6	CSPRNG (delegates to OS via <code>getrandom</code>)
zeroize	1.7	Memory zeroization
ed25519-dalek	2.1	Legacy Ed25519 (non-approved, gated)
sha2	0.10	Legacy SHA-256 (non-approved, gated)
hmac	0.12	Legacy HMAC (non-approved, gated)